

AI Umrah Companion — Technical Architecture & Build Plan

1. Overview

AI Umrah Companion is a multilingual, voice-first mobile application that assists pilgrims with disabilities in performing Umrah and Hajj rituals. The system combines on-device computer vision, spatial tracking, and cloud-based multimodal language models to deliver real-time environmental awareness, ritual state tracking, and adaptive guidance.

This document describes the technical architecture, component choices, and build sequence for the application.

2. System Architecture

The application follows a hybrid edge-cloud architecture. Latency-sensitive, privacy-sensitive, and connectivity-critical functions run on the device; reasoning-heavy and content-intensive functions run in the cloud.

On-device (mobile) - Camera frame capture and pre-processing - Object and pedestrian detection (YOLO via Core ML / TensorFlow Lite) - Depth sensing (LiDAR on supported iPhones; monocular depth estimation on other devices) - Visual-inertial spatial tracking (ARKit / ARCore) - Ritual state machine and lap counting - Speech-to-text (on-device Whisper variant via iOS Speech framework / Android SpeechRecognizer) - Text-to-speech playback - Local caching of ritual content and offline fallback logic

Cloud (backend) - Multimodal scene reasoning (Gemini 2.5 Flash or equivalent) - Companion matching service - User identity and ritual session persistence - Multilingual content delivery (sign-language video, simplified visual aids) - Analytics and telemetry

3. Mobile Application

Platforms: iOS (primary), Android (secondary). Both platforms are required for meaningful coverage of the pilgrim population.

Framework choice: Flutter is the recommended cross-platform framework for the application shell, because it provides consistent UI rendering, accessibility primitives, and integration with platform-native libraries. Native modules (Swift / Kotlin) handle the AR, camera, and on-device ML pipelines, exposed to Flutter via platform channels.

Why not React Native: Camera and AR performance on React Native depends on third-party bridges that lag platform releases. Flutter's lower-level rendering pipeline gives more predictable frame timing for camera-driven UIs.

Accessibility: The app must comply with WCAG 2.2 AA at minimum and exceed it for the disability-specific modes. VoiceOver and TalkBack support is required end-to-end; semantic labels are part of the design, not bolted on.

4. Vision Pipeline

The vision pipeline runs continuously while the app is in active assistance mode.

Frame capture

Component: AVFoundation (iOS) / CameraX (Android)

Runtime: On-device

Object detection

Component: YOLOv8 or YOLOv11, trained on COCO + custom pilgrim/ritual dataset, exported to Core ML / TFLite

Runtime: On-device, target 30 FPS

Depth

Component: LiDAR (iPhone Pro), ARKit depth API (iPhone 12+), ARCore Depth API (supported Android), Depth Anything V2 (fallback)

Runtime: On-device

Sensor fusion

Component: Custom layer that fuses bounding boxes with depth map to produce labelled distances

Runtime: On-device

Scene description

Component: Multimodal LLM (Gemini 2.5 Flash for cost-latency, Claude Sonnet 4.6 or GPT-4o for higher quality when needed)

Runtime: Cloud, called on demand

On-device first: The vision pipeline never sends a continuous video stream to the cloud. The on-device detector produces structured observations (object class, bounding box, distance). When the user asks an open-ended question (“what’s happening around me?”), a single still frame plus the structured observations is sent to the cloud LLM for scene-level reasoning. This bounds bandwidth, latency, privacy exposure, and cost.

Dataset: Public datasets (COCO, OpenImages) cover generic objects but not Hajj-specific entities (Kaaba, Black Stone corner, Maqam Ibrahim, Sa’i markers, ritual signage). A custom dataset must be assembled — annotated frames from publicly available Hajj footage and from on-site capture sessions — and used to fine-tune the YOLO model. This is a multi-month workstream.

5. Spatial Tracking and Ritual State Machine

Tawaf: The pilgrim walks seven counter-clockwise circuits around the Kaaba. The app must reliably count circuits across a range of walking speeds, with periodic pauses for prayer and supplication, and despite GPS being unreliable inside the Mataf (the marble area is partially covered and dense with metal structures).

Approach: ARKit (iOS) / ARCore (Android) provide visual-inertial odometry with sub-meter accuracy in feature-rich environments. The app establishes a local coordinate frame anchored at the start of Tawaf, tracks the pilgrim’s path, and applies a geometric heuristic: a circuit completes when the pilgrim’s accumulated angular displacement around the centroid of their initial trajectory exceeds 2π radians and they return within a threshold of their start point.

Sa’i: The pilgrim walks seven traversals between the hills of Safa and Marwah, a ~450m corridor. Lap counting here is simpler — detect direction changes and threshold crossings at the two endpoints.

Failure modes to handle: - ARKit drift in featureless or repetitive environments (the Mataf marble can confuse visual SLAM during peak crowds when feature points are occluded) - User pausing for prayer and resuming - User stepping out of the ritual path temporarily

Mitigation: Periodic anchor re-establishment, confidence scoring on each detected circuit, and a user-facing “I think you completed circuit 4, is that right?” affordance for edge cases.

6. Voice Interface

Languages (priority order): Arabic (Modern Standard + Hijazi dialect), Urdu, Indonesian, Malay, Turkish, English, Bengali, French. These eight cover roughly 80% of annual Hajj pilgrims by origin.

Speech-to-text: OpenAI Whisper (large-v3 or successor) for cloud transcription where quality matters; on-device equivalents (iOS Speech framework with Apple Intelligence on iPhone 15 Pro and newer, Android SpeechRecognizer) for offline operation. Word

error rates for Arabic dialects remain a known weakness — the system must gracefully handle low-confidence transcriptions by asking the user to repeat.

Text-to-speech: ElevenLabs for premium multilingual voice quality, with Apple AVSpeechSynthesizer and Android TTS as offline fallbacks. Voice persona is consistent across languages so the user experiences the same assistant identity.

Conversation management: The assistant maintains a short rolling context window (the last 10 exchanges plus ritual state) to handle follow-up questions naturally. The full conversation history is not sent to the LLM on every turn — only relevant state.

7. Adaptive User Interface

The application detects or is configured for the user's primary disability and adapts both input and output:

- **Blind / low-vision:** Voice-only, full screen reader compatibility, haptic feedback for ritual milestones, audio scene descriptions on demand.
- **Mobility-impaired:** Standard visual interface plus proactive crowd-density warnings and route suggestions.
- **Deaf / hard-of-hearing:** Visual notifications, captions for all spoken content, sign-language video clips (Arabic Sign Language, Indo-Pakistani Sign Language, International Sign) for key ritual steps.
- **Cognitive disabilities:** Simplified visual flowcharts, one-step-at-a-time interface, pictographic ritual progress, large clear buttons, no time pressure on any interaction.

Adaptive UI is not a single toggle; users may have intersecting needs (deaf-blind, mobility + cognitive) and the application supports independent toggles per modality.

8. Backend Services

Stack: Python + FastAPI for application logic; PostgreSQL for persistent state; Redis for ephemeral session state and companion matching; object storage (S3-compatible) for sign-language video clips and ritual content.

Hosting: Deployment must be in-Kingdom (Saudi Arabia) for regulatory compliance with KSA personal data protection law and for latency. Saudi cloud regions on AWS (Bahrain or Riyadh), Google Cloud (Dammam), or Oracle Cloud (Jeddah) are all options. Operationally, a single primary region with multi-AZ replication is sufficient; the load is regional, not global.

Companion matching service: A real-time service that matches solo pilgrims requesting assistance with nearby volunteer pilgrims who have opted in. Built on a simple geospatial index (PostGIS) plus opt-in availability flags. Volunteer onboarding includes identity verification through Absher or Nafath (the KSA national identity platforms).

Emergency signal: When a pilgrim triggers the panic button, the system shares live location with designated contacts and, if the user has opted in, with the Hajj security operations center via an integration with the General Directorate of Civil Defense or the Public Security Directorate. This integration requires formal agreement with KSA authorities and is a deployment dependency, not a build-time component.

9. Multimodal LLM Strategy

Multimodal LLM calls are the most expensive component of the system at scale. Cost discipline is structural.

Default model: Gemini 2.5 Flash for scene description and conversation. Low cost per call, good multilingual quality, fast.

Escalation model: Claude Sonnet 4.6 or GPT-4o for higher-quality reasoning when the Flash model returns low confidence or the user's question requires deeper context. The system measures and escalates automatically, not on every call.

On-device model: Apple Intelligence on-device models (iPhone 15 Pro+) handle ritual state reasoning ("am I done with circuit 4?") without any network call. This covers a significant fraction of conversational turns and reduces cloud cost meaningfully.

Prompt caching: All cloud LLM calls use the provider's prompt caching feature. The base system prompt (which contains ritual rules, multilingual phrasing guidance, and accessibility instructions) is large and stable — caching it cuts per-call cost by roughly 80%.

10. Connectivity and Offline Behavior

The Grand Mosque and surrounding pilgrim zones have congested WiFi during peak times. Cellular coverage is generally good but bandwidth contention is high. The application must remain useful under these conditions.

Always works offline: - Object detection, depth, and spatial tracking (all on-device) - Ritual lap counting and state machine - On-device voice for known commands and ritual narration - Cached ritual content (pre-loaded sign-language videos, pictographic flowcharts)

Requires connectivity (degrades gracefully): - Open-ended scene description via cloud LLM - Companion matching - Premium multilingual TTS (ElevenLabs); falls back to on-device TTS when offline - Cloud Whisper transcription; falls back to on-device speech recognition

The offline fallback is not an afterthought — it is the default mode in design reviews, and the cloud-enhanced experience is the upgrade.

11. Privacy and Data Protection

Default: No camera frames leave the device unless the user explicitly invokes a feature that requires cloud reasoning. Object detection output (class, bounding box, distance) leaves the device only when the user asks an open-ended question, and even then only a single annotated frame is sent.

Regulatory: KSA Personal Data Protection Law (PDPL) compliance is required. All personal data processed in-Kingdom; international transfers are restricted and only with explicit user consent.

Sensitive use case: This application processes data about pilgrims with disabilities — a sensitive category. Storage is minimized, retention is bounded, and analytics are aggregated and de-identified.

12. Performance Engineering

Latency budgets - On-device object detection: <50ms per frame - Ritual lap detection: <200ms from circuit completion to user notification - Voice question → spoken answer (cloud path): <2.5s end-to-end - Voice question → spoken answer (on-device path): <1s

Battery: The vision pipeline is power-hungry. Continuous camera + ML + cloud + display can drain a phone in ~90 minutes. Mitigations: duty-cycle the camera when the user is stationary, drop frame rate when no objects of interest are detected, use efficient model variants (YOLOv8n or YOLOv11n on lower-end devices), aggressive use of on-device models to avoid radio wakeups for LLM calls. The target is ≥4 hours of active use on a fully charged modern phone, with power-bank usage assumed for full ritual coverage.

Thermal: Sustained ML inference plus high ambient temperatures in Mecca (45°C+ in summer) will throttle phones. The application detects thermal state via OS APIs and reduces detection frame rate and cloud call frequency to stay below throttling thresholds.

13. Integration with KSA Pilgrim Infrastructure

Nusuk (the official KSA pilgrim platform): Integration via Nusuk's developer APIs to read the user's pilgrimage package, ritual schedule, and identity. This avoids duplicate onboarding and ensures the application is operating in sync with the user's official itinerary.

Tawakkalna: Integration with Tawakkalna for crowd density data where available. Public APIs for crowd density at the Grand Mosque are limited; partnership with the Ministry of Hajj and Umrah may unlock more granular data feeds.

Absher / Nafath: National identity verification for the companion matching feature; volunteers must be verified to participate.

These integrations are partnership-dependent and require formal agreements with the relevant KSA authorities. They are not blockers for core functionality but are required for the application to be officially endorsed and distributed at scale.

14. Build Sequence

Phase 1 — Core technical proof (4–6 weeks) - iOS-only application shell in Flutter with native modules for AR and camera - On-device YOLO object detection with COCO model (no custom Hajj entities yet) - ARKit-based Tawaf lap counting demonstrated in a controlled environment - Voice loop: Whisper STT → Gemini 2.5 Flash → ElevenLabs TTS, in Arabic and English - Single ritual (Tawaf), single disability modality (blind / low-vision), end-to-end

Phase 2 — Multi-ritual, multi-modality (6–10 weeks) - Sa'i ritual support - Mobility, deaf, and cognitive disability modes - Android port with ARCore parity - Custom Hajj-specific object detection dataset and fine-tuned model - Sign-language video content (Arabic Sign Language) for ritual steps

Phase 3 — Backend and integrations (8–12 weeks) - Companion matching service - Nusuk and Tawakkalna integrations - Identity verification via Absher / Nafath - In-Kingdom hosting and PDPL compliance review - Pilot deployment with 100–500 volunteer pilgrims during a low-season Umrah window

Phase 4 — Hajj-readiness (12+ weeks) - Load testing for peak Hajj concurrency (target: 100,000 concurrent active users) - Thermal and battery optimization for July–August deployment - Emergency signal integration with KSA security infrastructure - Multilingual expansion beyond the initial eight languages - Formal accessibility audits by KSA disability advocacy organizations

15. Open Technical Questions

These are decisions deferred pending more information:

- Whether to invest in a custom multilingual TTS voice model versus relying on ElevenLabs at scale (cost vs. control trade-off)
- Whether to build crowd density estimation from on-device camera + crowd-sourced aggregation, or to depend exclusively on official feeds from Tawakkalna and the Ministry of Hajj
- Whether to support smart-glasses form factors (Meta Ray-Ban, Apple Vision Pro variants) as a second client surface, or to remain phone-only for the foreseeable future
- The right balance of on-device fine-tuned LLM versus cloud LLM for ritual state reasoning, as on-device model capability continues to improve